LAB-2 Assignment
Date: 11/08/2026
Topic: Shell

Q1.2 (Predict the output) — Ordering without wait()

Given:

```
pid_t pid = fork();
if (pid == 0)
    printf("I am child. PID = %d\n", getpid());
else
    printf("I am parent. PID = %d\n", getpid());
```

Will "I am child" always print before "I am parent"? Why or why not?

Ans: no i am a child will not always print before i am a parent . this is because of the cpu scheduling based on how cpu decide to schedule child running depends on which algorithm we or os is using.

Q1.3 (Predict the output) — How many "Hello"s?

How many times does "Hello\n" get printed by this program?

```
int main() {
    fork();
    fork();
    printf("Hello\n");
    return 0;
}
```

Ans: 4 times as the fork creates below program 2 times and the another fork will create more 2 times so there will be at the end 4 printf commands.

```
👉 ❯ ./new
Hello
Hello
Hello
Hello
```

Q2 (Code) — Write the child/parent branching pattern:
Write the skeleton of a program that forks, and in the child prints "I am child. PID = %d\n", in the parent prints "I am parent. PID = %d\n", and handles fork failure.

Code:

```
int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    } else if (pid == 0) {
        printf("I am child. PID = %d\n", getpid());
    } else {
        printf("I am parent. PID = %d\n", getpid());
        wait(NULL);
    }
    return 0;
}
```

Output:

```
 ❯ ./Q2
I am parent. PID = 53135
I am child. PID = 53136
```

Q3: (Code) - runcmd.c (fork + exec + wait combined)

Write a program that takes exactly two command-line arguments — a command and one argument to it (e.g. sleep 10) — forks a child, execs the command in the child, and has the parent wait, reap, and report success. It must print an error for the wrong number of arguments.

Hints to write the program: Your program needs to do **four main things**:

1. **Check command-line arguments**
    - argc tells you how many arguments were supplied
2. Create a child process
    - fork() has three possible results:
        - < 0 → process creation failed
        - == 0 → you are inside the **child**
        - > 0 → you are inside the **parent**
3. In the child, execute the requested command
4. In the parent, wait for the child

Code:

runcmd.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <command> <arg>\n", argv[0]);
        return 1;
    }
    pid_t pid = fork();
    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0) {
        execlp(argv[1], argv[1], argv[2], NULL);
        printf("Command execution failed!\n");
        exit(1);
    }
    else {
        wait(NULL);
        printf("Child process completed successfully.\n");
    }
    return 0;
}
```

Output:

```
harshal@harshalhp in ~/course/os/Lab-2 via C v16.1.1-gcc
 ❯ ./runcmd ls -l
total 48
-rw-r--r-- 1 harshal harshal   139 Aug 11 12:06 new.c
-rwxr-xr-x 1 harshal harshal 16216 Aug 11 12:14 Q2
-rw-r--r-- 1 harshal harshal   408 Aug 11 12:18 Q2.c
-rwxr-xr-x 1 harshal harshal 16216 Aug 11 12:27 runcmd
-rw-r--r-- 1 harshal harshal   576 Aug 11 12:27 runcmd.c
-rw-r--r-- 1 harshal harshal   120 Aug 11 19:08 shell.c
Child process completed successfully.

harshal@harshalhp in ~/course/os/Lab-2 via C v16.1.1-gcc
 ❯
```

Q4 (Code):Background execution:

Now, we will extend the shell to support background execution of processes. Extend your shell program of part A in the following manner: if a Linux command is followed by &, the command must be executed in the background. That is, the shell must start the execution of the command, and return to prompt the user for the next input,

without waiting for the previous command to complete. The output of the command can get printed to the shell as and when it appears. A command not followed by & must simply execute in the foreground as before.

You can assume that the commands running in the background are simple Linux commands without pipes or redirections or any other special case handling like cd. You can assume that the user will enter only one foreground or background command at a time on the command prompt, and the command and & are separated by a space. You may assume that there are no more than 64 background commands executing at any given time. A helpful tip for testing: use long running commands like sleep to test your foreground and background implementations, as such commands will give you enough time to run ps in another window to check that the commands are executing as specified. Across both background and foreground execution, ensure that the shell reaps all its children that have terminated. Unlike in the case of foreground execution, the background processes can be reaped with a time delay. For example, the shell may check for dead children periodically, say, when it obtains a new user input from the terminal. When the shell reaps a terminated background process, it must print a message Shell: Background process finished to let the user know that a background process has finished. Hint: you may use a variant of the wait system call, e.g., waitpid(-1, NULL, WNOHANG) to reap background processes without blocking. Read up more on how to invoke wait without blocking.

You must test your implementation for the cases where background and foreground processes are running together, and ensure that dead children are being reaped correctly in such cases. Recall that a generic wait system call can reap and return any dead child. So if you are waiting for a foreground process to terminate and invoke wait, it may reap and return a terminated background process. In that case, you must not erroneously return to the command prompt for the next command, but you must wait for the foreground command to terminate as well. To avoid such confusions, you may choose to use the waitpid variant of this system call, to be sure that you are reaping the correct foreground child. Once again, use long running commands like sleep, run ps in another window, and monitor the execution of your processes, to thoroughly test your shell with a combination of background and foreground processes. In particular, test that a background process finishing up in the middle of a foreground command execution will not cause your shell to incorrectly return to the command prompt before the foreground command finishes.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
```

```c
#define MAX_INPUT_LEN 1024
#define MAX_ARGS 64

/*
* Reaps any terminated background processes without blocking.
* Prints "Shell: Background process finished" for each reaped process.
*/
void reap_background_processes(void) {
   pid_t pid;
   int status;
   while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
       printf("Shell: Background process finished\n");
       fflush(stdout);
   }
}

/*
* Returns the number of arguments (excluding trailing NULL and "&").
*/
int parse_line(char *line, char **args, int *is_background) {
   int count = 0;
   char *token = strtok(line, " \t\r\n");
   while (token != NULL && count < MAX_ARGS - 1) {
       args[count++] = token;
       token = strtok(NULL, " \t\r\n");
   }
   args[count] = NULL;
   *is_background = 0;

   if (count > 0 && strcmp(args[count - 1], "&") == 0) {
       *is_background = 1;
       args[count - 1] = NULL;
       count--;
   }
   return count;
}

int main(void) {
   char line[MAX_INPUT_LEN];
   char *args[MAX_ARGS];
   int is_background = 0;

   while (1) {
```

```
        // Reap any terminated background processes before prompting
        reap_background_processes();

        // Display prompt
        printf("shell> ");
        fflush(stdout);

        // Read user input
        if (fgets(line, sizeof(line), stdin) == NULL) {
            if (feof(stdin)) {
                // EOF (Ctrl+D)
                printf("\n");
                break;
            }
            if (ferror(stdin) && errno == EINTR) {
                clearerr(stdin);
                printf("\n");
                continue;
            }
            break;
        }

        // Parse line into arguments
        int num_args = parse_line(line, args, &is_background);
        if (num_args == 0) {
            continue;
        }

        // Handle shell built-in commands
        if (strcmp(args[0], "exit") == 0) {
            reap_background_processes();
            break;
        }

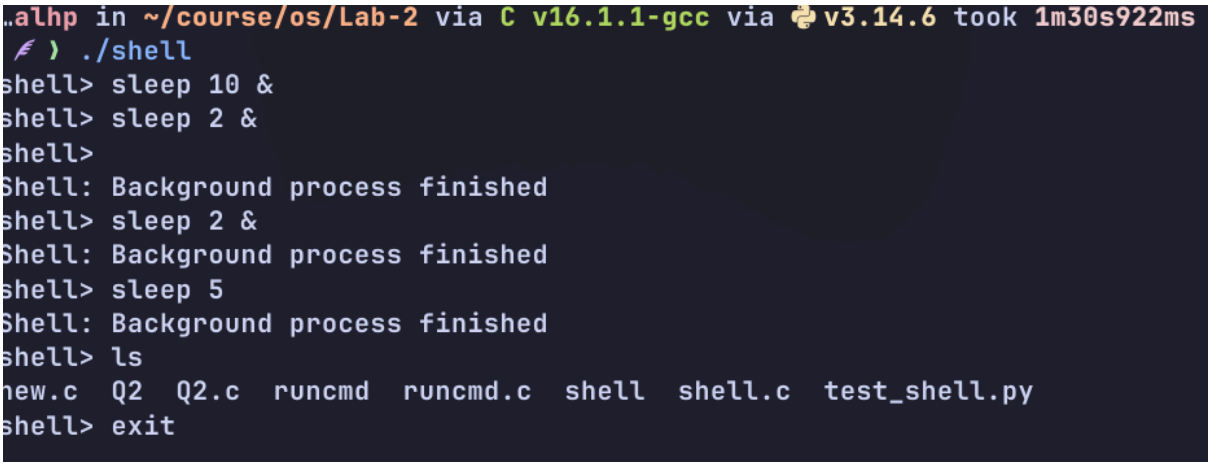
        if (strcmp(args[0], "cd") == 0) {
            if (args[1] == NULL || strcmp(args[1], "~") == 0) {
                char *home = getenv("HOME");
                if (home != NULL) {
                    if (chdir(home) != 0) {
                        perror("cd failed");
                    }
                }
            } else {
```

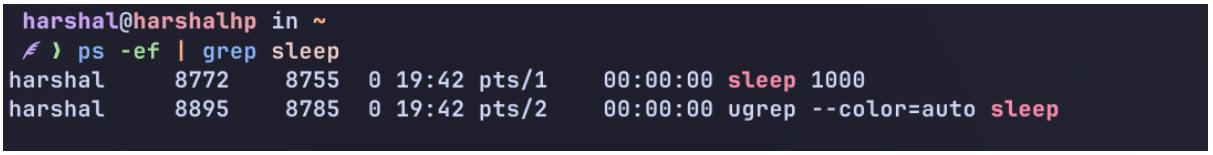
```
            if (chdir(args[1]) != 0) {
                perror("cd failed");
            }
        }
        continue;
    }

    // Fork child process for command execution
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
    } else if (pid == 0) {
        // Child Process: execute command
        if (execvp(args[0], args) < 0) {
            perror("Execution failed");
            exit(EXIT_FAILURE);
        }
    } else {
        // Parent Process
        if (is_background) {
            // Background execution: do not wait, prompt user
immediately
        } else {
            // Foreground execution: wait specifically for this
foreground child PID
            int status;
            while (waitpid(pid, &status, 0) == -1) {
                if (errno != EINTR) {
                    perror("waitpid failed");
                    break;
                }
            }
            // Reap any background processes that completed during
foreground execution
            reap_background_processes();
        }
    }
}

return 0;
}
```

Output:


…alhp in ~/course/os/Lab-2 via C v16.1.1-gcc via v3.14.6 took 1m30s922ms
./shell
shell> sleep 10 &
shell> sleep 2 &
shell>
Shell: Background process finished
shell> sleep 2 &
Shell: Background process finished
shell> sleep 5
Shell: Background process finished
shell> ls
new.c Q2 Q2.c runcmd runcmd.c shell shell.c test_shell.py
shell> exit


harshal@harshalhp in ~
ps -ef | grep sleep
harshal 8772 8755 0 19:42 pts/1 00:00:00 sleep 1000
harshal 8895 8785 0 19:42 pts/2 00:00:00 ugrep --color=auto sleep


…shalhp in ~/course/os/Lab-2 via C v16.1.1-gcc via v3.14.6 took 37s935ms
./shell
shell> sleep 1000 &
shell>